Reference lecture (click)

1 Brute Force Classification

1.1 Supervised Machine Learning

- Input space X, Output space Y
- Training data: given many instances $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$
- Learn a function $X \to Y$ which is expressed by parameters
- Test data: Given new $\mathbf{x}_{N+1}$, use function to predict y_{N+1}
- Evaluation: Check whether prediction y_{N+1} matches the ground truth y^*_{N+1}
- y^*_{N+1} is not known to the black box (held-out)

1.2 Text Categorization

- Given a text document, assign it to one of K categories.
- Consider how you're deciding the category and how you would explain it to the computer.
- Input space: documents, Output: one of K categories
- Training data: documents and their category (e.g. sports, news)
- Learn a categorization function
- Test data: Given a new document, use function to predict category
- Evaluation: Check whether predicted category is correct
- Evaluation measure: how many correct predictions?

1.2.1 Binary Text Categorization Task

- There is only one category.
- Predict whether the document is in the category or not (pos/neg)
- Same: Predict the set of documents that are in this category.
- Input space: documents, Output: $\{0, 1\}$ (1=positive)
- Training data: documents and whether they are in this category (e.g. sports or not)
- Learn a categorization function
- Evaluation measure: how many correct predictions?

1.2.2 One-vs-Rest Classification

- Input space: documents, Output: one of K categories
- Redefine the task to learn K binary categorization functions
- Learn K categorization functions (one per category)
- Can a document be positive for multiple categories? If yes, which category to choose?
  - Each categorization function bases the decision on a **Classification Score** or **Confidence**
  - Assign the document to the category with the highest confidence

1.2.3 Supervised Document Classification

- Represent documents with feature vectors
- Example content features:
  - $f_1 d$: 1 if doc d contains the word "ball"; 0 otherwise
  - $f_2(d)$: 1 if doc d contains the world "goal"; 0 otherwise
  - $f_3(d)$: 1 if doc d contains the word "election"; 0 otherwise
- Change the parameters in the function to get more predictions right
- Iteratively update parameters (Training loop)
- Then fix the parameters, make one prediction in the test set
- Sometimes: very good on the train set, but errors on the test set
- **Note: Never train on the test data!**
  - Never test on the training data (unless you call it training error)
  - Never leak the test data into your training process

1.3 A Simple Categorization Model

1.3.1 Linear Hyperplane Model

- Feature vectors in space X (number of features = dimension of space)
- Find a hyperplane that separates positive from negative examples
- To classify a new example, see if it is on the "positive" side of the hyperplane
  - Weight vector $\mathbf{w}$ points towards positive side
- *The hyperplane has one fewer dimension than the vector space*

1.3.2 Hyperplane Parameters

- Hyperplane is represented by vector $\mathbf{w}$
- $\mathbf{w}$ is orthogonal to the hyperplane
- $\mathbf{w}$ has the same dimensionality as $\mathbf{x}_i$
- If $\mathbf{w} \cdot \mathbf{x}_i > 0$ then example i is positive
- Inner product $\mathbf{w} \cdot \mathbf{x}_i = \sum_j w_j x_{ij}$ over feature dimensions j.

1.3.3 Inner Products and Cosines

- If $\mathbf{w} \cdot \mathbf{x} > 0$ then example i is positive
- Same as: $\angle(\mathbf{w}, \mathbf{x}_i) < 90\,\text{deg}$
- Same as: $\cos(\mathbf{w}, \mathbf{x}_i) > 0$ $\quad$ $\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}||\,||\mathbf{b}||}$

1.3.4 Hard vs Soft Classification

- Sometimes we want to tolerate classification errors in training data to get a more robust hyperplane
- We don't want outliers and errors to dictate our model.
- **Hard classification** is strict; the hyperplane exactly and entirely divides the space into positive and negative
- **Soft classification** allows for some errors, typically to allow for a simpler hyperplane. This also prevents outliers and errors from causing issues with classification

1.4 The Intercept of the Model

1.4.1 Changing the Threshold with "The Intercept"

- Changing the threshold b: $\mathbf{w} \cdot \mathbf{x}_1 > b$ $\quad$ The intercept b
- By adjusting b, you shift the threshold of classification.
- **Intercept trick:** It is easy to forget to add the intercept for $\mathbf{w} \cdot \mathbf{x}_i > b$.
  - Rewrite as $\mathbf{w} \cdot \mathbf{x}_i + (-b) \cdot 1 > 0$, recognize the inner products
  - Consider the feature vector $\mathbf{x}_i$ with D dimensions. Append one additional feature dimension as follows:

$$\mathbf{x}_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \dots \\ x_{iD} \\ 1 \end{pmatrix} \qquad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_D \\ -b \end{pmatrix}$$

  - Train a classification model on augmented $\mathbf{x}$ without intercept:
    - Classify: If $\mathbf{w} \cdot \mathbf{x}_i > 0$ then i is positive

- Automatically learns the intercept as $w_{D+1} = -b$

- **Note:** For the rest of the course, we will use the intercept trick. Whenever you load features x_i, don't forget to add another 1 as a constant feature.

1.5 Training – Coordinate Ascent

If $\mathbf{w} \cdot \mathbf{x}_i > 0$ then example i is positive.

- Start with an initial $\mathbf{w}$, which can be vector of zeros or random.
- Repeat: Randomly select a dimension j (coordinate)
  - Vary the entry w_j up or down, observe classification quality
  - Lock in the best setting for w_j
  - Move onto next dimension...
- Keep on repeating until the change in quality is small
- To "Vary w_j": use a grid of values Δ_k to try out, e.g.

$$\begin{pmatrix} w_1 \\ ... \\ w_j \\ ... \\ w_{D+1} \end{pmatrix} \rightarrow \begin{pmatrix} w_1 \\ ... \\ w_j + \Delta_1 \\ ... \\ w_{D+1} \end{pmatrix} \rightarrow \begin{pmatrix} w_1 \\ ... \\ w_j + \Delta_2 \\ ... \\ w_{D+1} \end{pmatrix} \rightarrow ...$$

- Use each $\mathbf{w}$ to classify all training examples, choose $\mathbf{w}$ with the best prediction quality (before advancing to next j).
- **When to stop?:** Since changing w_1 will affect the best choice for w_2, etc. we have to consider the same coordinate j multiple times.
- There might always be a tiny improvement, loop forever...
- Run until the relative improvement in quality is small. How to measure quality? $\rightarrow$

1.6 How to measure Prediction Quality?

- "choose $\mathbf{w}$ with the best prediction quality"
- **Accuracy:** how many examples i are correctly predicted (higher is better)
- **Misclassification rate:** fraction of mistakes (lower is better)
- We can easily use other measures of prediction quality
  - **Precision:** among all i that are predicted as positive, how many are correct?
  - **Recall:** of all actually positive i, how many are predicted as positive?
  - **F1-Measure:** harmonic mean of precision and recall

$$\mathrm{F}_1 \text{ score } = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Others:
  - ROC-AUC ("Area under the Receiver-Operator Curve")
  - For ranking problems: MAP, NDCG, Precision-at-R, Utility

1.6.1 Confusion Matrix

- For binary classification, Precision, Recall, F1, Accuracy are based on the **confusion matrix**.
  - $S = \{i \text{ that are predicted as positive}\}$
  - $S^* = \{i \text{ that are actually positive}\}$
  - U: Set of all instances i.

The Confusion Matrix

	predicted positive S	predicted negative $U \setminus S$
Actual positive S^*	true positives (TP) $S \cap S^*$	false negatives (FN) $S^* \setminus S$
Actual negative $U \setminus S^*$	false positives (FP) $S \setminus S^*$	true negatives (TN) $U \setminus (S \cup S^*)$

1.7 Coordinate Ascent with Features and Z-Score Normalization

1.7.1 Coordinate Ascent Recap/Recontextualize

Start with a random $\mathbf{w}^{(0)}$ (**initialization**).
Iterate many times (for t in 1...): (**outer loop**)

- Randomly select a dimension j (**coordinate**)
  - For every Δ_k on the grid: (**inner loop**)
    - Temporary vector $\tilde{\mathbf{w}}_{jk} = \mathbf{w}^{(t-1)} + \Delta_k \mathbf{1}_j$
    - Note: $\mathbf{1}_j = (0, ..., 0, 1, 0, ...)$ is the **unit vector** for coordinate j
    - **Prediction quality** $\tilde{O}_k$ = classification quality when using $\tilde{\mathbf{w}}_{jk}$.
  - Identify **best step size** $k_{\text{best}} = \arg\max_k \tilde{O}_k$
  - **Update the weight vector** $\mathbf{w}^{(t)} = \tilde{\mathbf{w}}_{jk_{\text{best}}}$
  - **Quality of iteration** t is $O^{(t)} = \tilde{(O)}_{k_{\text{best}}}$

Keep on iterating until the relative change in quality O is small.

- E.g.: Define convergence as improvement over previous 100 iterations

$$1 - \frac{O^{t-100}}{O^{(t)}} < 0.01$$

- Stop iteration when converged (maybe plot convergence over t)

Will the quality O always improve over time?
Yes! Well: it will never decrease (if you include step size $\Delta_{\text{stay}} = 0$).

1.7.2 Features

- Features common in text categorization are contained words
- Example content features:

  $x_{i1} = f_1(d)$: 1 if doc d contains the word "ball"; 0 otherwise
  $x_{i2} = f_2(d)$: 1 if doc d contains the word "goal"; 0 otherwise
  $x_{i3} = f_3(d)$: 1 if doc d contains the word "election"; 0 otherwise
  ...
  $x_{i(V+1)} = f_{V+1}(d) = 1$ (intercept !!)

1.7.3 Z-Score Normalization

Z-Score normalization re-centers the feature vectors so that they have the same mean and variance. The grid needs to be chosen to explore the range of the distribution.

- We transform all feature vectors, so that
  - mean μ_j across examples $\mathbf{x}_i$ will be zero
  - the standard deviation σ_j across examples $\mathbf{x}_i$ will be one
- This is called an **isotropic Gaussian distribution** $\mathcal{N}(0, 1)$
- For each feature j:
  - compute its mean μ_j across examples $\mathbf{x}_i$ (using only j'th entries)
  - compute its standard deviation σ_j across examples $\mathbf{x}_i$
- Transform all features of all feature vectors $\mathbf{x}_i$ via

$$x_{ij} := \frac{x_{ij} - \mu_j}{\sigma_j}$$

1.7.4 Term-Frequency Inverse-Document-Frequency (TF IDF)

- Represent all documents d_i in a corpus
- Vector space $\mathfrak{F} = \mathcal{R}^V$: one dimension per word type ("ball")

- Term Frequency: $\text{tf}_{j,i}$ is how often word j appears in doc d_i.
- The j'th entry of document feature $\mathbf{x}_i = x_{ij}$ is

$$x_{ij} = \underbrace{\log(1 + \text{tf}_{j,i})}_{\text{TF}} \cdot \underbrace{\log(\frac{\text{tf}_{*,*}}{1 + \text{tf}_{j,*}})}_{\text{IDF}}$$

$$\text{tf}_{j,*} = \Sigma_{\text{docs } i'} \ \text{tf}_{j,i'}$$

$$\text{tf}_{*,*} = \Sigma_{\text{words } j'} \Sigma_{\text{docs } i'} \ \text{tf}_{j',i'}$$